

LAB 03 – TRANG INDEX VÀ ABOUT

3.1. TẠO TRANG INDEX VÀ ABOUT

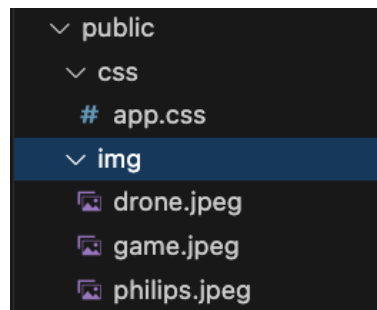
3.1.1. INDEX VIEW

Trang chủ hiện chỉ hiển thị thông báo chào mừng. Hãy tạo view index mới. Trong resources/views/, tạo thư mục con home. Trong resources/views/home, tạo một tập tin mới index.blade.php và thêm code sau vào đó.

Code

```
@extends('layouts.app')
@section('title', $viewData["title"])
@section('content')
<div class="row">
  <div class="col-md-6 col-lg-4 mb-2">
    
  </div>
  <div class="col-md-6 col-lg-4 mb-2">
     </div>
  <div class="col-md-6 col-lg-4 mb-2">
    
  </div>
</div>
@endsection
```

@section đầu tiên chèn nội dung của biến viewData["title"] vào. Biến đó sẽ được xác định trong tập tin route web và được chuyển đến view này. Sau đó, chúng ta xác định một số phân chia để hiển thị một số hình ảnh. Chúng ta cần tải xuống những hình ảnh này và lưu trữ chúng trong thư mục public. Đầu tiên, trong thư mục public, tạo thư mục con img. Sau đó, tải xuống ba hình ảnh và lưu trữ chúng bên trong thư mục public/img (xem Hình 3-1).



Hình 3-1. Cấu trúc dự án khi lưu trữ hình ảnh.

3.1.2. ABOUT VIEW

Hãy tạo view About. Trong resources/views/home, tạo một tập tin mới about.blade.php và

thêm code sau vào đó.

Code

```
@extends('layouts.app')
@section('title', $title)
@section('subtitle', $subtitle)
@section('content')
    <div class="container">
        <div class="row">
            <div class="col-lg-4 ms-auto">
                <p class="lead">{{ $description }}</p>
            </div>
            <div class="col-lg-4 me-auto">
                <p class="lead">{{ $author }}</p>
            </div>
        </div>
    </div>
@endsection
```

Chúng ta có một view đơn giản hiển thị mô tả về ứng dụng và một số thông tin về tác giả. Chúng ta sẽ chuyển bốn biến từ controllers sang view này sau. Hãy nhớ rằng Blade cho phép các dấu ngoặc nhọn hiển thị dữ liệu được truyền vào view.

3.2. GIỚI THIỆU ĐỊNH TUYẾN LARAVEL

Định tuyến Laravel là một cơ chế được sử dụng để định tuyến tất cả các yêu cầu ứng dụng đến các phương thức hoặc chức năng cụ thể sẽ xử lý các yêu cầu cụ thể đó. Các tuyến Laravel chấp nhận URI (Uniform Resource Identifier) cùng với closure. Các closures là phiên bản của các hàm ẩn danh của PHP. Một closure là một hàm mà có thể truyền dưới dạng đối tượng, gán cho một biến hoặc truyền dưới dạng tham số cho các hàm và phương thức khác.

Các tuyến của Laravel được xác định trong các tập tin tuyến (nằm trong thư mục routes).

1. Tập tin routes/web.php xác định các tuyến cho giao diện web. Đây là những tuyến mà chúng ta sẽ sử dụng trong hướng dẫn này.
2. Tập tin routes/api.php xác định các tuyến cho API (nếu có). Đây là các tuyến được sử dụng trong kiến trúc hướng dịch vụ hoặc API REST.

Hãy kiểm tra một vài tuyến để hiểu cách chúng hoạt động. Trong routes/web.php, những thay đổi trong phần in đậm (màu đỏ).

Code

```
<?php
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
```

```

return view('welcome');
$viewData = [];
$viewData["title"] = "Trang chu - Online Store";
return view('home.index')->with("viewData", $viewData);
});

Route::get('/about',
'App\Http\Controllers\HomeController@about')->name("home.about");

```

Trong đoạn code trên, chúng ta đã trình bày hai cách xác định tuyến trong Laravel.

Tuyến đầu tiên kết nối URI “/” với một bao đóng trả về một view (trong trường hợp này là view home.index).

view() là một trình trợ giúp của Laravel để truy xuất một phiên bản view. Kiểm tra cách chúng ta chuyển biến viewData sang view home.index bằng cách kết nối phương thức with với phương thức trợ giúp view.

Tuyến thứ hai kết nối URI “/about” với phương thức HomeController about (được tạo sau). Ngoài ra, chúng ta xác định tên tuyến tùy chỉnh bằng cách xâu chuỗi phương thức name vào định nghĩa tuyến.

3.3. LARAVEL CONTROLLERS

Việc xác định tất cả logic xử lý yêu cầu bên trong các phần đóng của tập tin route có vẻ không thông minh. Sẽ dẫn đến hàng trăm hoặc hàng nghìn dòng code bên trong các tập tin tuyến (điều này ảnh hưởng đến khả năng bảo trì dự án). Một chiến lược tốt là tổ chức hành vi này bằng cách sử dụng các lớp “controller”. Controllers có thể nhóm logic xử lý yêu cầu liên quan vào một lớp duy nhất. Ví dụ: lớp UserController có thể xử lý tất cả các yêu cầu đến liên quan đến người dùng, bao gồm hiển thị, tạo, cập nhật và xóa người dùng.

Trong app/Http/Controllers, tạo một tập tin mới HomeController.php và thêm code sau vào tập tin đó.

Code

```

<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;

class HomeController extends Controller {
    public function index()
    {
        $viewData = [];
        $viewData["title"] = "Trang chu - Online Store";
        return view('home.index')->with("viewData", $viewData);
    }
}

```

```
public function about()
{
    $data1 = "Gioi thieu - Online Store";
    $data2 = "Gioi thieu";
    $description = "Day la trang about gioi thieu!";
    $author = "Phat trien boi: OnlyU.";
    return view('home.about')
        ->with("title", $data1)
        ->with("subtitle", $data2)
        ->with("description", $description)
        ->with("author", $author);
}
```

Chúng ta có HomeController mở rộng lớp Controller Laravel. Chúng ta có phương thức index và phương thức about được kết nối với tuyến ("/about") trong tập tin routes/web.php. Phương thức này xác định một tập hợp các biến và chuyển chúng tới view home.about.

Tóm lại, khi người dùng đi đến tuyến ("/"), view home.index sẽ được hiển thị (được gửi trong tập tin route/web.php). Tương tự như vậy, khi người dùng đi đến tuyến ("/about"), view home.about sẽ được hiển thị (được phân phối bởi phương thức about trong HomeController).

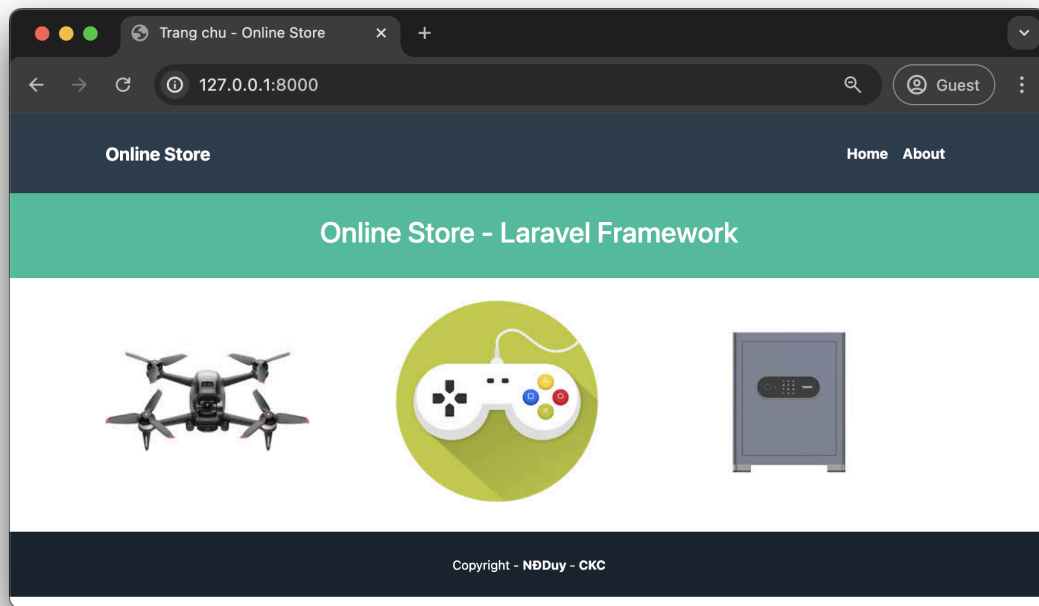
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

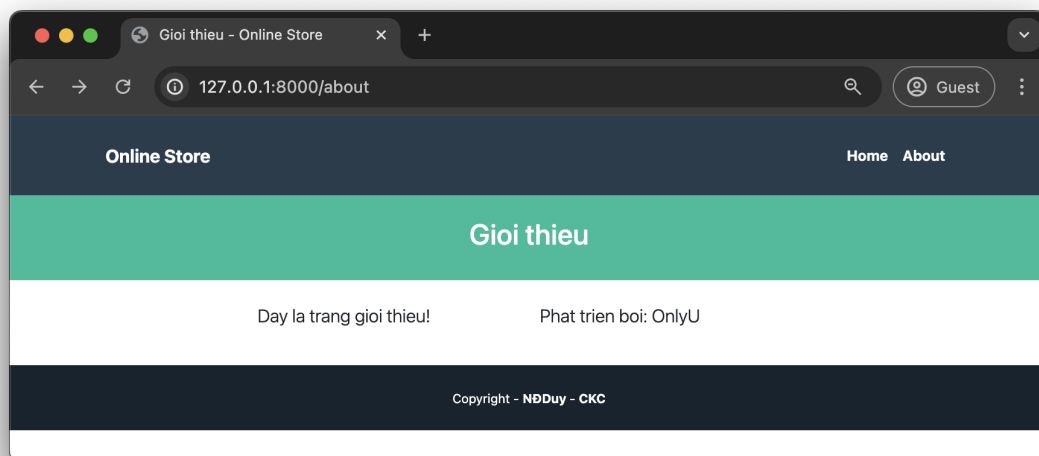
Terminal

```
php artisan serve
```

Đi tới tuyến ("/") và sẽ thấy trang chủ mới (xem Hình 3-2). Sau đó, đi tới tuyến ("/about") và sẽ thấy trang about (xem Hình 3-3).



Hình 3-2. Online Store – Home page.



Hình 3-3. Online Store – About page.

3.4. TÁI CẤU TRÚC TRANG INDEX VÀ ABOUT

Code trong phần trước có thể được làm sạch và cải thiện hơn nữa. Chúng ta chưa xác định quy tắc code hóa bất kỳ phần tử nào trước đó. Chúng ta sẽ xem cách cấu trúc lại code để làm cho code sạch hơn và dễ bảo trì hơn. Chúng ta cũng sẽ cung cấp các mẹo chung để xử lý các tuyến, controllers và view. Hãy nhớ rằng hầu hết các nguyên tắc được trình bày trong phần này này không chỉ áp dụng cho dự án Laravel mà còn có thể được áp dụng trong các khung MVC khác (chẳng hạn như Django, Spring, Nest, Express, v.v.).

3.4.1. TÁI CẤU TRÚC CÁC TUYẾN

Các tuyến trước đó

Phần trước đã trình bày hai tuyến được xác định bằng các cách tiếp cận khác nhau. Hãy kiểm tra cái đầu tiên.

Code

```
Route::get('/', function () {  
    $viewData = [];  
    $viewData["title"] = "Trang chu - Online Store";  
    return view('home.index')->with("viewData", $viewData);  
});
```

Đó là một trong những cách tiếp cận tồi tệ nhất mà có thể áp dụng (bất kể đang sử dụng framework nào). Nếu đặt logic ứng dụng vào tập tin tuyến, sẽ kết thúc với hàng trăm hoặc hàng nghìn dòng code trong các tập tin đó khi dự án phát triển và khả năng bảo trì sẽ trở thành địa ngục. Vì vậy, chúng ta không nên sử dụng chiến lược này ngay cả đối với các dự án nhỏ.

Bây giờ, hãy phân tích cái thứ hai.

Code

```
Route::get('/about',  
'App\Http\Controllers\HomeController@about')->name("home.about");
```

Nó trông tốt hơn. Tuyến của chúng ta hiện được kết nối với một phương thức điều khiển. Chúng ta cũng đặt tên cho tuyến đó. Chúng ta nên kết hợp tên của controllers với tên phương thức của controllers để xác định tên tuyến.

Routes mới

Hãy cấu trúc lại các tuyến của chúng ta. Trong routes/web.php, hãy xóa tuyến ("/") trước đó và thay thế nó bằng tuyến in đậm (màu đỏ) sau.

Code

```
Route::get('/', function () {  
    {  
        $viewData = [];  
        $viewData["title"] = "Home Page - Online Store";  
        return view('home.index')->with("viewData", $viewData);  
    }  
});  
  
Route::get('/', 'App\Http\Controllers\HomeController@index')->name("home.index");
```

```
Route::get('/about', 'App \ Http \ Controllers \
HomeController@about')->name("home.about");
```

Bây giờ, các tuyến web trông gọn gàng và nhất quán hơn.

Với tư cách là nhà phát triển phần mềm, một chiến lược tốt là tạo một tài liệu có các quy tắc kiến trúc và chia sẻ tài liệu đó với nhóm. Có thể tạo tài liệu đó trong kho dự án (nếu có). Khuyến khích tất cả các thành viên đọc tài liệu đó. Quy tắc đầu tiên mà có thể đưa vào tài liệu đó có thể là: "Trong các tập tin tuyến, mọi URI chỉ được kết nối với các phương thức điều khiển. KHÔNG được phép đưa logic ứng dụng vào trong tập tin tuyến". Những quy tắc đơn giản này sẽ giúp tiết kiệm được nhiều thời gian và không phải đau đầu. Chúng ta luôn tạo một tài liệu như vậy cho tất cả các dự án và chúng ta khuyến khích làm điều đó trong các dự án.

3.4.2. TÁI CẤU TRÚC CONTROLLERS

Controllers trước đó

Trước đó đã có một controllers với hai phương thức sử dụng các cách tiếp cận khác nhau.

Hãy kiểm tra phương thức about.

Code

```
public function about()
{
    $data1 = "Gioi thieu - Online Store";
    $data2 = "Gioi thieu";
    $description = "Day la trang gioi thieu!";
    $author = "Phat trien boi: OnlyU.";
    return view('home.about')
        ->with("title", $data1)
        ->with("subtitle", $data2)
        ->with("description", $description)
        ->with("author", $author);
}
```

Ở đây chúng ta có ba vấn đề.

1. Đặt tên biến là một mớ hỗn độn. Sử dụng những cái tên như data1 hay data2 thật kinh khủng, nó chẳng nói lên điều gì. Thay vào đó, chúng ta có thể sử dụng tiêu đề và phụ đề.
2. Chúng ta có nhiều phương thức bị xiềng xích. Hãy tưởng tượng nếu chúng ta có 20 biến để truyền vào view. Chúng ta không có sự nhất quán. Chúng ta gửi từng biến tới view lần lượt, nhưng chúng ta chỉ chuyển một biến mảng cho view trong phương thức index. Chúng ta thích chiến lược phương thức index hơn vì chúng ta sẽ thấy nó tiếp theo.
3. Cuối cùng, chúng ta có một dòng trống trước khi kết thúc dấu ngoặc nhọn. Chúng ta cần xác định một hướng dẫn về phong cách code hóa nhất quán.

Hãy xem tầm quan trọng của việc đặt tên biến bằng hai trích dẫn từ cuốn sách (2019 - Thomas, D., & Hunt, A. - The Pragmatic Programmer: your Journey to Mastery). "Sự khởi đầu của trí tuệ là khả năng gọi đúng tên của sự vật. - Nho giáo." - "Tại sao việc đặt tên lại quan trọng? Bởi vì những cái tên hay làm cho code dễ đọc hơn và bạn phải đọc nó để thay đổi nó."

Bây giờ, hãy phân tích phương thức index (đã bị bỏ qua ở phần trước).

Code

```
public function index()
{
    $viewData = [];
    $viewData["title"] = "Trang chu - Online Store";
    return view('home.index')->with("viewData", $viewData);
}
```

Trong trường hợp này, chúng ta chỉ xác định một biến gọi là viewData chứa tất cả dữ liệu được gửi đến view. Biến này là một mảng kết hợp. Với cách tiếp cận này, việc chúng ta chuyển một dữ liệu đến view hay hàng chục dữ liệu đều không thành vấn đề. Trong cả hai trường hợp, chúng ta chỉ truyền mảng kết hợp.

Trong tài liệu quy tắc kiến trúc, có thể bao gồm một quy tắc mới nói rằng controllers chỉ nên chuyển một mảng kết hợp có tên viewData cho các view. Laravel cung cấp nhiều cách để truyền dữ liệu đó tới view, việc chọn cách nào không quan trọng. Tuy nhiên, điều cần thiết là phải đề cập đến cách tiếp cận đã chọn trong tài liệu quy tắc để tất cả các thành viên trong nhóm sử dụng nhất quán thông qua ứng dụng.

Controller mới

Hãy cấu trúc lại controllers. Trong app/Http/Controllers/HomeController.php, hãy xem những thay đổi in đậm (màu đỏ) sau.

Code

```
<?php

namespace App\Http\Controllers;
use Illuminate\Http\Request;

class HomeController extends Controller
{
    public function index()
    {
        $viewData = [];
        $viewData["title"] = "Trang chu - Online Store";
        return view('home.index')->with("viewData", $viewData);
    }
}
```



```

public function about()
{
    $viewData = [];
    $viewData["title"] = "Gioi thieu - Online Store";
    $viewData["subtitle"] = "Gioi thieu";
    $viewData["description"] = "Day la trang gioi thieu!";
    $viewData["author"] = "Phat trien boi: OnlyU";
    return view('home.about')->with("viewData", $viewData);
}
}

```

Bây giờ chúng ta sử dụng chiến lược biến đơn, cả hai phương pháp đều nhất quán. Chúng ta nên lưu trữ các view theo cách tiếp cận tiếp theo. Đầu tiên, chúng ta tạo một thư mục con có tên của controllers bên trong thư mục resources/views. Trong trường hợp này, thư mục con được gọi là home. Và nếu một phương thức của controllers hiển thị dữ liệu, chúng ta sẽ tạo một tập tin có tên phương thức của controllers bên trong thư mục con của view tương ứng với controller. Vì vậy, chúng ta có tập tin about.blade.php được lưu trữ bên trong thư mục con resources/views/home. Việc tìm kiếm các view cụ thể cho các phương thức điều khiển cụ thể giúp dễ dàng hơn.

3.4.3. TÁI CẤU TRÚC VIEWS

Phần trước đã trình bày hai view hiển thị dữ liệu hơi khác một chút. View home/index sẽ không bị thay đổi vì nó hiển thị dữ liệu bằng chiến lược viewData. Tuy nhiên, chúng ta sẽ cần sửa đổi view home/about để phù hợp với chiến lược biến đơn được xác định trước đó.

Hãy cấu trúc lại views about. Trong resources/views/home/about.blade.php, hãy thay đổi phần in đậm (màu đỏ) sau.

Code

```

@extends('layouts.app')
@section('title', $viewData["title"])
@section('subtitle', $viewData["subtitle"])
@section('content')
<div class="container">
    <div class="row">
        <div class="col-lg-4 ms-auto">
            <p class="lead">{{ $viewData["description"] }}</p>
        </div>
        <div class="col-lg-4 me-auto">
            <p class="lead">{{ $viewData["author"] }}</p>
        </div>
    </div>
</div>
@endsection

```

Như có thể thấy, bây giờ chúng ta truy cập dữ liệu thông qua mảng kết hợp viewData duy nhất. Chúng ta sẽ sử dụng chiến lược này trên toàn bộ ứng dụng, giúp quan điểm của chúng ta nhất quán hơn.

Dữ liệu trước đó cũng có thể được đặt trực tiếp lên các view trước đó. Ý là không cần xác định những văn bản đó dưới dạng các biến trong controllers và gửi chúng đến view. Thay vào đó, có thể đặt văn bản trực tiếp trong view như chúng ta đã làm trong view Welcome.blade.php. Chúng ta đã làm theo cách đó để minh họa và giải thích một số thành phần của Laravel. Tùy chọn đó được gọi là Bản địa hóa (Localization). Trong Bản địa hóa, migrations các văn bản đó khỏi controllers và view rồi đặt chúng vào thư mục resources/lang. Bản địa hóa cho phép thậm chí truy xuất các chuỗi bằng nhiều ngôn ngữ khác nhau. Nó không khó để thực hiện. Để biết thêm thông tin về nó <https://laravel.com/docs/11.x/localization#introduction>, hoặc tìm kiếm trên Google.

3.4.4. CẬP NHẬT LINKS TRONG HEADER

Bây giờ chúng ta đã có các tuyến, controllers và view thích hợp, hãy đưa các liên kết vào tiêu đề. Trong resources/views/layouts/app.blade.php, hãy thay đổi phần in đậm (màu đỏ).

Code

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous" />
    <link href="{{asset('/css/app.css')}}" rel="stylesheet"/>
    <title>@yield('title', 'Online Store')</title>
</head>
<body>
    <!-- header -->
    <nav class="navbar navbar-expand-lg navbar-dark bg-secondary py-4">
        <div class="container">
            <a class="navbar-brand" href="{{ route('home.index') }}">Online
Store</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse"
data-bs-target="#navbarNavAltMarkup" aria-controls="navbarNavAltMarkup"
aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNavAltMarkup">
                <div class="navbar-nav ms-auto">
                    <a class="nav-link active" href="{{ route('home.index')
}}">>Home</a>
                    <a class="nav-link active" href="{{ route('home.about')
}}">>About</a>
```

```

        </div>
    </div>
</div>
</nav>
<header class="masthead bg-primary text-white text-center py-4">
    <div class="container d-flex align-items-center flex-column">
        <h2>@yield('subtitle', 'Online Store - Laravel Framework')</h2>
    </div>
</header>
<!-- header -->
<div class="container my-4">
    @yield('content')
</div>
<!-- footer -->
<div class="copyright py-4 text-center text-white">
    <div class="container">
        <small> Copyright - <a class="text-reset fw-bold text-decoration-none"
target="_blank" href="https://twitter.com/">
            NĐDuy
            </a> - <b>CKC</b>
        </small>
    </div>
</div>
<!-- footer -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.1.3/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous">
    </script>
</body>
</html>

```

Chúng ta sử dụng phương thức route helper trong bố cục trước đó để tạo URL cho tuyến được đặt tên nhất định. Chúng ta đã sử dụng tên của các tuyến được xác định cho tuyến ("/") (home.index) và tuyến ("/about") (home.about).

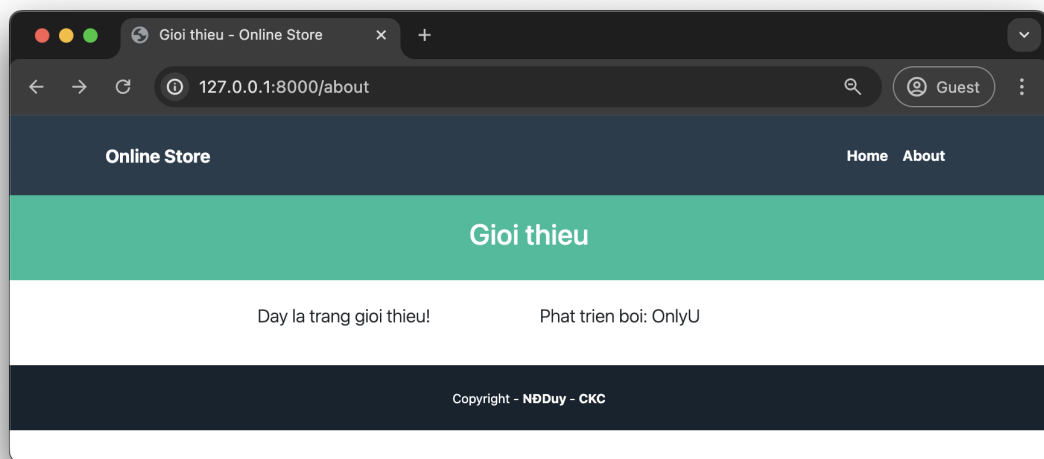
Chạy ứng dụng

Trong Terminal, đi tới thư mục dự án và thực hiện như sau:

Terminal

```
php artisan serve
```

Bây giờ, có thể điều hướng giữa Home và trang About bằng cách sử dụng các liên kết trong thanh điều hướng (xem Hình 3-4).



Hình 3-4. Online Store – About page.